

Complete Guide: TypeScript Services & Controllers

Table of Contents

- 1. [Services - Complete Guide](#)
- 2. [Interfaces - When to Use](#)
- 3. [Parameters - How to Identify](#)
- 4. [Quick Reference](#)

Services - Complete Guide {#services}

What IS a Service?

A service is a class (or object) that contains your **business logic**. It:

- Talks to your database
- Does calculations
- Validates things
- Does NOT know about HTTP requests/responses

Step 1: Basic Service Structure



typescript

```
// src/services/user.service.ts

class UserService {
  // Methods go here
}

export default new UserService();
```

That's it. A class with methods. Export one instance.

Step 2: Add Simple Methods



typescript

```
class UserService {
  async getAllUsers() {
    // Logic to get users from DB
    return [
      { id: 1, name: "Alice" },
      { id: 2, name: "Bob" }
    ];
  }

  async getUserById(id: number) {
    // Logic to get one user
    return { id, name: "Alice" };
  }
}

export default new UserService();
```

Notice:

- Methods are async (because DB calls are async)
 - They return data, not HTTP responses
 - Simple, readable names
-

Step 3: Add Types (TypeScript)

Create Types File:



typescript

```
// src/types/user.types.ts
```

```
export interface User {
  id: number;
  name: string;
  email: string;
}

export interface CreateUserDto {
  name: string;
  email: string;
}
```

Update Service:



typescript

```
// src/services/user.service.ts
import { User, CreateUserDto } from '../types/user.types';

class UserService {
  async getAllUsers(): Promise<User[]> {
    return [
      { id: 1, name: "Alice", email: "alice@example.com" }
    ];
  }

  async getUserById(id: number): Promise<User | null> {
    return { id, name: "Alice", email: "alice@example.com" };
  }

  async createUser(data: CreateUserDto): Promise<User> {
    const newUser = {
      id: Date.now(),
      ...data
    };
    return newUser;
  }
}

export default new UserService();
```

Key Points:

- Promise<User[]> - returns array of users
- Promise<User | null> - might return null
- CreateUserDto - type for input data

Step 4: Add Database (Real Logic)



typescript

```
import { User, CreateUserDto } from '../types/user.types';
import prisma from '../db'; // or your DB client

class UserService {
  async getAllUsers(): Promise<User[]> {
    const users = await prisma.user.findMany();
    return users;
  }

  async getUserById(id: number): Promise<User | null> {
    const user = await prisma.user.findUnique({
      where: { id }
    });
    return user;
  }

  async createUser(data: CreateUserDto): Promise<User> {
    const user = await prisma.user.create({ data });
    return user;
  }

  async updateUser(id: number, data: Partial<CreateUserDto>): Promise<User> {
    const user = await prisma.user.update({
      where: { id },
      data
    });
    return user;
  }

  async deleteUser(id: number): Promise<void> {
    await prisma.user.delete({
      where: { id }
    });
  }
}

export default new UserService();
```

Step 5: Add Error Handling



typescript

```

class UserService {
  async getUserId(id: number): Promise<User | null> {
    try {
      const user = await prisma.user.findUnique({
        where: { id }
      });

      if (!user) {
        return null; // Let controller handle 404
      }

      return user;
    } catch (error) {
      console.error('Error fetching user:', error);
      throw new Error('Failed to fetch user');
    }
  }
}

```

Step 6: Advanced - Validation



typescript

```

async createUser(data: CreateUserDto): Promise<User> {
  // Check if email already exists
  const existingUser = await prisma.user.findUnique({
    where: { email: data.email }
  });

  if (existingUser) {
    throw new Error('Email already in use');
  }

  const user = await prisma.user.create({ data });
  return user;
}

```

Step 7: Advanced - Related Data



typescript

```
async getUserWithPosts(id: number): Promise<User & { posts: Post[] } | null> {
  const user = await prisma.user.findUnique({
    where: { id },
    include: {
      posts: true
    }
  });

  return user;
}
```

Complete Service Template



typescript

```
import { User, CreateUserDto } from '../types/user.types';
import prisma from '../db';

class UserService {
  // GET ALL
  async getAllUsers(): Promise<User[]> {
    return await prisma.user.findMany();
  }

  // GET ONE
  async getUserById(id: number): Promise<User | null> {
    return await prisma.user.findUnique({ where: { id } });
  }

  // CREATE
  async createUser(data: CreateUserDto): Promise<User> {
    return await prisma.user.create({ data });
  }

  // UPDATE
  async updateUser(id: number, data: Partial<CreateUserDto>): Promise<User> {
    return await prisma.user.update({ where: { id }, data });
  }

  // DELETE
  async deleteUser(id: number): Promise<void> {
    await prisma.user.delete({ where: { id } });
  }
}

export default new UserService();
```

Interfaces - When to Use {#interfaces}

What IS an Interface?

An interface is a **contract** or **blueprint** that describes the shape of data.



typescript

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
}
```

This says: "A User object MUST have these 3 properties with these types."

WHEN TO USE Interfaces

1. For Data Models (Database entities)



typescript

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
  createdAt: Date;  
}
```

✔ Use when: Representing data from your database

2. For Input Data (DTOs)



typescript

```
interface CreateUserDto {  
  name: string;  
  email: string;  
  password: string;  
}  
  
interface UpdateUserDto {  
  name?: string; // Optional  
  email?: string;  
}
```

✔ Use when: Data coming FROM the user (request body)

3. For Function Return Types



typescript

```
interface LoginResponse {  
  token: string;  
  user: User;  
}  
  
async login(): Promise<LoginResponse> {  
  // ...  
}
```

✔ Use when: You want to be clear what the function returns

4. For Complex Objects



typescript

```
interface UserWithPosts {  
  id: number;  
  name: string;  
  posts: Post[];  
}
```

✔ Use when: Combining multiple types or adding relations

WHEN NOT TO USE Interfaces

1. Simple Primitive Types



typescript

// ❌ DON'T DO THIS

```
interface UserId {  
  value: number;  
}
```

// ✅ DO THIS

```
type UserId = number;  
// or just use: number directly
```

2. When Type is Obvious



typescript

// ❌ Unnecessary

```
async deleteUser(id: number): Promise<void> {  
  // void is obvious  
}
```

// ✅ Clean

```
async deleteUser(id: number) {  
  // TypeScript infers Promise<void>  
}
```

3. For Simple Arrays



typescript

// ❌ Overkill

```
interface UserIds {  
  ids: number[];  
}
```

// ✅ Simple

```
const ids: number[] = [1, 2, 3];
```

Quick Rule

- Use **interface** when data has **multiple properties**
- **Don't use** for single values or when type is obvious

Parameters - How to Identify {#parameters}

The Question to Ask

"What information does this function NEED to work?"

Example 1: Get All Users



typescript

```
async getAllUsers() {  
  // Needs: NOTHING (getting everything)  
  return await prisma.user.findMany();  
}
```

✔ No parameters needed

Example 2: Get One User



typescript

```
async getUserById(id: number) {  
  // Needs: Which user? → ID  
  return await prisma.user.findUnique({ where: { id } });  
}
```

✔ One parameter: id (to identify which user)

Example 3: Create User



typescript

```
async createUser(data: CreateUserDto) {  
  // Needs: What data to create? → name, email, password  
  return await prisma.user.create({ data });  
}
```

✔ **One parameter:** data object (all the info needed)

Why not separate parameters?



typescript

```
// ✗ Too many parameters (messy)
async createUser(name: string, email: string, password: string) { }
```



```
// ✔ Clean - group related data
async createUser(data: CreateUserData) { }
```

Example 4: Update User



typescript

```
async updateUser(id: number, data: UpdateUserData) {
  // Needs:
  // 1. Which user? → ID
  // 2. What to change? → data
  return await prisma.user.update({ where: { id }, data });
}
```

✔ **Two parameters:**

- id - to identify the user
 - data - what to update
-

Example 5: Search Users



typescript

```

async searchUsers(query: string, limit?: number) {
  // Needs:
  // 1. What to search? → query
  // 2. How many results? → limit (optional)

  return await prisma.user.findMany({
    where: { name: { contains: query } },
    take: limit || 10
  });
}

```

✅ Parameters:

- query - required
- limit - optional (? means optional)

Parameter Types

Type 1: Identifiers



typescript

```

id: number
userId: string
email: string

```

Use when: Finding/updating/deleting **specific** item

Type 2: Data Objects



typescript

```

data: CreateUserDto
data: UpdateUserDto

```

Use when: Creating or updating (multiple fields)

Type 3: Filters/Options



typescript

```
filters: { status?: string; role?: string }
options: { limit?: number; offset?: number }
```

Use when: Searching or listing with conditions

Type 4: Simple Values



typescript

```
isActive: boolean
page: number
sortBy: string
```

Use when: Single config value

Complete Example: Product Service



typescript

```
interface Product {  
  id: number;  
  name: string;  
  price: number;  
  stock: number;  
}
```

```
interface CreateProductDto {  
  name: string;  
  price: number;  
  stock: number;  
}
```

```
class ProductService {  
  // Get all - NO parameters  
  async getAllProducts(): Promise<Product[]> {  
    return await prisma.product.findMany();  
  }  
  
  // Get one - NEEDS id  
  async getProductById(id: number): Promise<Product | null> {  
    return await prisma.product.findUnique({ where: { id } });  
  }  
  
  // Create - NEEDS data  
  async createProduct(data: CreateProductDto): Promise<Product> {  
    return await prisma.product.create({ data });  
  }  
  
  // Update - NEEDS id AND data  
  async updateProduct(  
    id: number,  
    data: Partial<CreateProductDto>  
  ): Promise<Product> {  
    return await prisma.product.update({ where: { id }, data });  
  }  
  
  // Delete - NEEDS id  
  async deleteProduct(id: number): Promise<void> {  
    await prisma.product.delete({ where: { id } });  
  }  
  
  // Search - NEEDS query and optional limit  
  async searchProducts(query: string, limit: number = 10): Promise<Product[]> {
```

```

return await prisma.product.findMany({
  where: { name: { contains: query } },
  take: limit
});
}

```

// Filter - NEEDS min/max

```

async getProductsByPriceRange(
  minPrice: number,
  maxPrice: number
): Promise<Product[]> {
  return await prisma.product.findMany({
    where: {
      price: { gte: minPrice, lte: maxPrice }
    }
  });
}
}

```

How to Think About Parameters

Ask yourself 3 questions:

1. "What am I trying to do?"
 - Get all → no parameters
 - Get specific → need ID
 - Create → need data
 - Update → need ID + data
2. "What information is REQUIRED?"
 - Can't work without it → required parameter
 - Nice to have → optional parameter ?
3. "Should I group parameters?"
 - 1-2 simple values → separate parameters
 - 3+ related values → group in object/interface

Quick Reference {#quick-reference}

Service Function Patterns

Function Type	Parameters	Example
Get All	None	getAllUsers()
Get One	ID	getUserById(id: number)
Create	Data object	createUser(data: CreateUserDto)
Update	ID + Data	updateUser(id: number, data: UpdateUserDto)
Delete	ID	deleteUser(id: number)
Search	Query + Options	searchUsers(query: string, limit?: number)
Filter	Filter values	getUsersByRole(role: string)

Interface Usage

Action	Parameters	Use Interface?
Create	Multiple fields	☒ YES
Update	Multiple fields	☒ YES
Get by ID	One value (ID)	☒ NO - just id: number
Filter by one thing	One value	☒ NO - just the value
Filter by many things	Multiple values	☒ YES

Simple Rules Summary

Services:

- ☒ Contain business logic
- ☒ Talk to database
- ☒ Return data
- ☒ Don't know about HTTP

Interfaces:

- ☒ Use for multiple properties
- ☒ Use for DTOs (Create/Update)
- ☒ Use for complex returns
- ☒ Don't use for single values

Parameters:

- ☒ Create/Update = Use interface (multiple fields)
- ☒ Get/Filter one thing = Use simple type
- ☒ Filter many things = Use interface

Examples Checklist

When writing a service method:

- ☐ What does it do? (Get/Create/Update/Delete/Search)
- ☐ What data does it need?
- ☐ Is it one value or multiple? (Interface or simple type?)
- ☐ What does it return? (Single/Array/Null?)
- ☐ Add error handling
- ☐ Add validation if needed

End of Guide

Convert this markdown to PDF using:

- VS Code: Right-click → "Markdown PDF: Export (pdf)"
- Online: dillinger.io, markdown-pdf.com
- Command line: `pandoc guide.md -o guide.pdf`